Author：Heath Huang(黄施泓) heath.huang@quectel.com
日期:2022.06.20

下面是基于 SDK: LTE01R06A02_C_SDK_A_BETA0602_SDK 做的 RAM 和 ROM 分析
首先 过滤下 kernel.map：搜索下关键字：Execution Region 或者 Max,如下：

```
E:\firewarm\all_in_one\ASR1606\LTE01R06A02_C_SDK_A_BETA0602_SDK\ql-sdk\ql-kernel\threadx\build\kernel.map （匹配45次）
行 53386:     Execution Region DDR_ITCM (Base: 0x00000000, Size: 0x0000d804, Max: 0x00010000, ABSOLUTE)
行 53781:     Execution Region DDR_DSP_RO (Base: 0x7e000000, Size: 0x00000000, Max: 0x00070000, ABSOLUTE)
行 53783:     **** No section assigned to this execution region ****
行 53786:     Execution Region DDR_DSP_RO_ENDMARK (Base: 0x7e06fffc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 53793:     Execution Region DDR_HEAP_GUARD (Base: 0x7e070000, Size: 0x001afffc, Max: 0x001afffc, ABSOLUTE)
行 53800:     Execution Region DDR_HEAP_ENDMARK (Base: 0x7e21fffc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 53807:     Execution Region APP_CODE_IN_PSRAM (Base: 0x7e220000, Size: 0x0027fffc, Max: 0x0027fffc, ABSOLUTE)
行 53814:     Execution Region APP_CODE_ENDMARK (Base: 0x7e49fffc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 53821:     Execution Region DDR_RW_DATA (Base: 0x7e4a0000, Size: 0x001496ec, Max: 0x001e0000, ABSOLUTE)
行 55476:     Execution Region DDR_RW_DATA_ENDMARK (Base: 0x7e67fffc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 55483:     Execution Region PS_CODE_IN_PSRAM (Base: 0x7e680000, Size: 0x0003de3c, Max: 0x0007fffc, ABSOLUTE)
行 56967:     Execution Region PS_CODE_IN_PSRAM_ENDMARKER (Base: 0x7e6fffc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 56974:     Execution Region DDR_NONCACHE_REGION_ACIPC_PS_UL (Base: 0x7e700000, Size: 0x0003e7fc, Max: 0x0003e7fc, ABSOLUTE)
行 56981:     Execution Region DDR_NONCACHE_REGION_ACIPC_PS_UL_ENDMARKER (Base: 0x7e73e7fc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 56988:     Execution Region DDR_NONCACHE_REGION_ACIPC_PS_DL (Base: 0x7e73e800, Size: 0x00018ffc, Max: 0x00018ffc, ABSOLUTE)
行 56995:     Execution Region DDR_NONCACHE_REGION_ACIPC_PS_DL_ENDMARKER (Base: 0x7e7577fc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 57002:     Execution Region PS_NONCACHE_DATA (Base: 0x7e757800, Size: 0x00069b20, Max: 0x0006fffc, ABSOLUTE)
行 57029:     Execution Region PS_NONCACHE_DATA_ENDMARK (Base: 0x7e7c77fc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 57036:     Execution Region DDR_NONCACHE_REGION (Base: 0x7e7c7800, Size: 0x00000f08, Max: 0x00000ffc, ABSOLUTE)
行 57052:     Execution Region DDR_NONCACHE_REGION_ENDMARKER (Base: 0x7e7c87fc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 57059:     Execution Region D2_VECT (Base: 0x7e7c8800, Size: 0x00000040, Max: 0x00000100, ABSOLUTE)
行 57066:     Execution Region DDR_PRIVATE_RW_AREA (Base: 0x7e7c8900, Size: 0x00001a24, Max: 0x00001afc, ABSOLUTE)
行 57079:     Execution Region DDR_PRIVATE_RW_AREA_ENDMARK (Base: 0x7e7ca3fc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 57086:     Execution Region DDR_NONCACHE_REGION_PTABLE (Base: 0x7e7ca400, Size: 0x00000ffc, Max: 0x00000ffc, ABSOLUTE)
行 57093:     Execution Region DDR_NONCACHE_REGION_PTABLE_ENDMARK (Base: 0x7e7cb3fc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 57100:     Execution Region DDR_NONCACHE_REGION_PROP (Base: 0x7e7cb400, Size: 0x00000ffc, Max: 0x00000ffc, ABSOLUTE)
行 57107:     Execution Region DDR_NONCACHE_REGION_PROP_ENDMARK (Base: 0x7e7cc3fc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 57114:     Execution Region DDR_NONCACHE_USB (Base: 0x7e7cc400, Size: 0x00023b00, Max: 0x00033c00, ABSOLUTE)
行 57146:     Execution Region PSRAM_END (Base: 0x7e7ffffc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 57153:     Execution Region DDR_RO_INIT_CODE (Base: 0x80022000, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 57160:     Execution Region LOAD_TABLE (Base: 0x80022004, Size: 0x0000057c, Max: 0x000011fc, ABSOLUTE, FIXED)
行 57167:     Execution Region BOARD_CONFIG (Base: 0x80023200, Size: 0x0000007c, Max: 0x00000300, ABSOLUTE, FIXED)
行 57174:     Execution Region PS_CONFIG (Base: 0x80023500, Size: 0x00000014, Max: 0x00000100, ABSOLUTE, FIXED)
行 57181:     Execution Region NVMBUF (Base: 0x80023600, Size: 0x00000008, Max: 0x00008000, ABSOLUTE, FIXED)
行 57188:     Execution Region DDR_SYS_RO_EXEC (Base: 0x80023608, Size: 0x00288f4c, Max: 0xffffffff, ABSOLUTE)
行 60228:     Execution Region DDR_RO_EXEC (Base: 0x802ac554, Size: 0x001840d0, Max: 0xffffffff, ABSOLUTE, FIXED)
行 61840:     Execution Region DDR_DTCM (Base: 0xb0020000, Size: 0x0000bb60, Max: 0x00010000, ABSOLUTE)
行 61908:     Execution Region DDR_DTCM_ENDMARKER (Base: 0xb002fffc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)
行 61915:     Execution Region USB_POOL (Base: 0xd1001000, Size: 0x00002000, Max: 0x00002000, ABSOLUTE)
行 61922:     Execution Region USB_POOL_DQH (Base: 0xd1004800, Size: 0x00000800, Max: 0x00000800, ABSOLUTE)
行 61929:     Execution Region DSP_SHM (Base: 0xd1005000, Size: 0x00000000, Max: 0xffffffff, ABSOLUTE)
行 61931:     **** No section assigned to this execution region ****
行 61934:     Execution Region DSP_BUF (Base: 0xd1005000, Size: 0x00000000, Max: 0xffffffff, ABSOLUTE)
行 61936:     **** No section assigned to this execution region ****
行 61939:     Execution Region LTEBMBLOCK_BUF (Base: 0xd1007000, Size: 0x00000f78, Max: 0x00001000, ABSOLUTE)
```

总的 max 举个例子比如 DDR_HEAP_GUARD--0x001afffc+0x00000004(对应的 ENDMARK)= 0x001b0000

```
Execution Region DDR_HEAP_GUARD (Base: 0x7e070000, Size: 0x001afffc, Max: 0x001afffc, ABSOLUTE)

Base Addr    Size         Type   Attr      Idx    E Section Name        Object

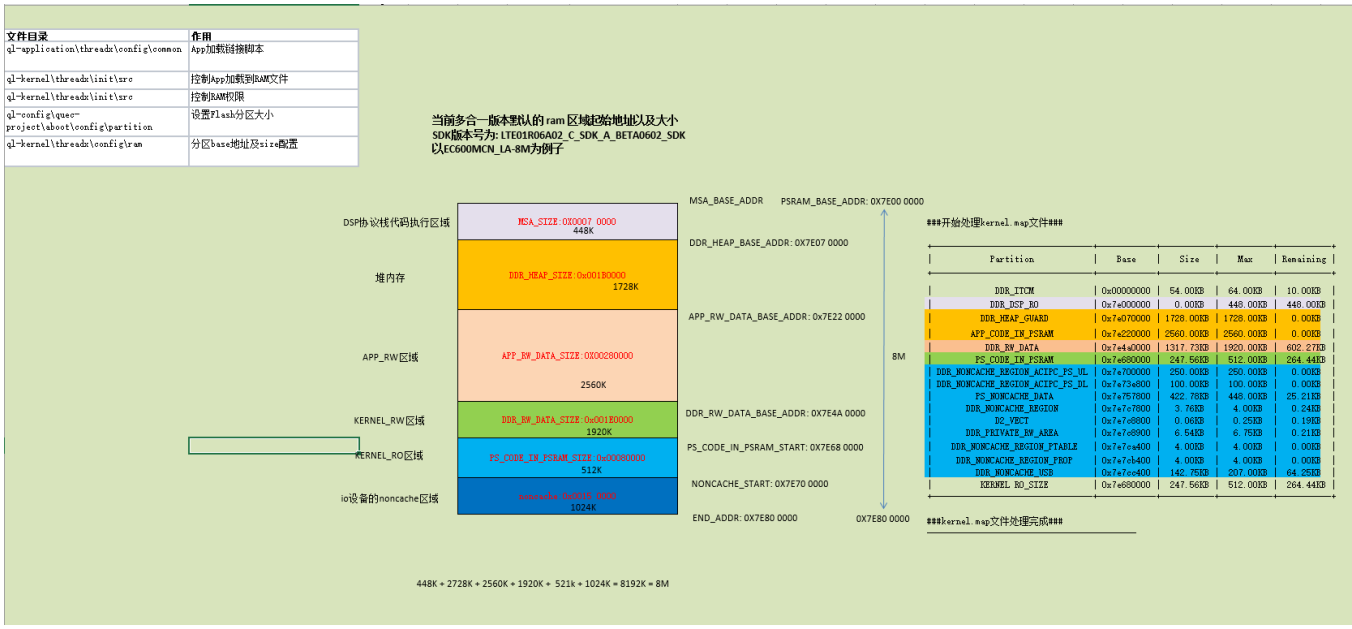
0x7e070000   0x001afffc   Zero   RW            3    DDR_HEAP_GUARD.bss  anon$$obj.o


Execution Region DDR_HEAP_ENDMARK (Base: 0x7e21fffc, Size: 0x00000004, Max: 0x00000004, ABSOLUTE)

Base Addr    Size         Type   Attr      Idx    E Section Name        Object

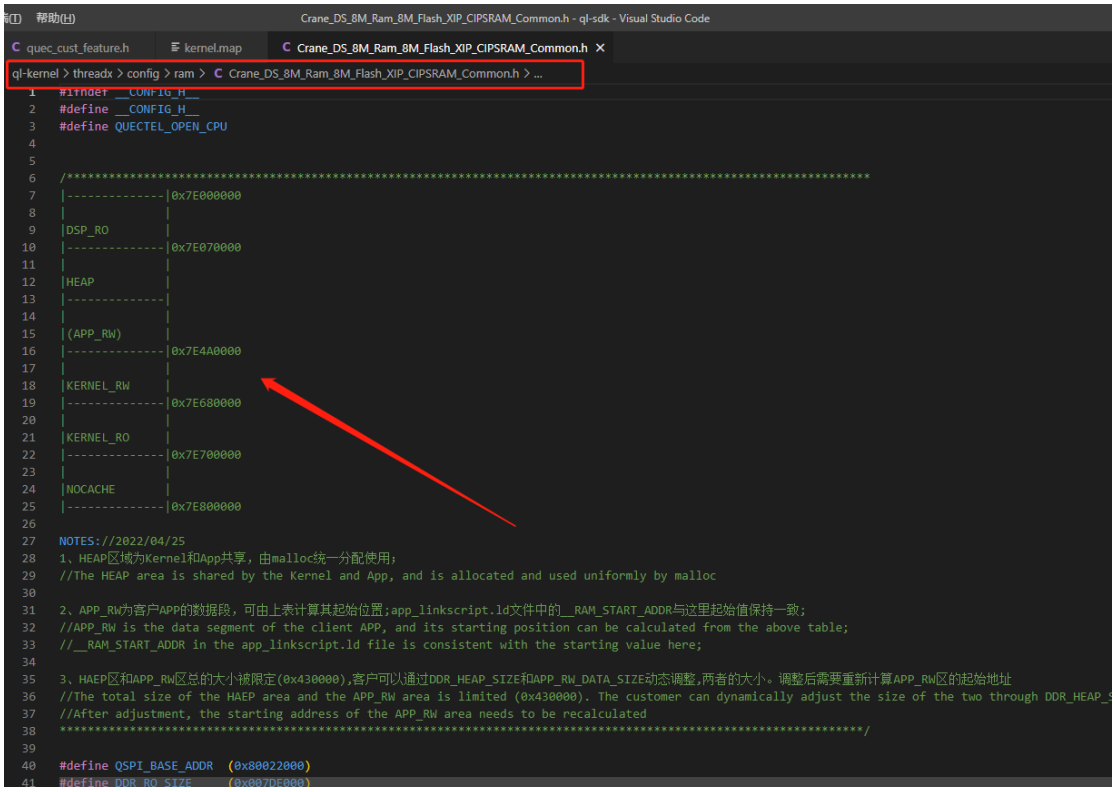
0x7e21fffc   0x00000004   Zero   RW            4    DDR_HEAP_ENDMARK.bss  anon$$obj.o
```

	base	use_size	Max
DDR_DSP_RO	0x7e000000	/	0x00070000
DDR_HEAP_GUARD	0x7e070000	0x001afffc	0x001b0000
DDR_RW_DATA	0x7e4a0000	0x001496ec	0x001e0000
PS_CODE_IN_PSRAM	0x7e680000	0x0003de3c	0x00080000
DDR_NONCACHE_REGION_ACIPC_PS_UL	0x7e700000	0x0003e7fc	0x0003e800



1606
RAM分布图.xlsx

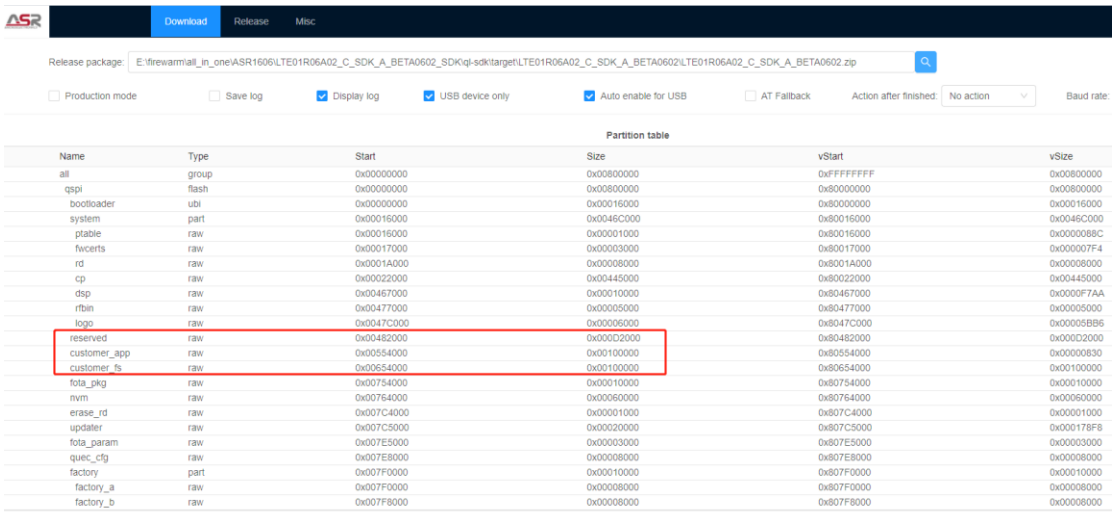
在对应型号比如 ql-kernel\threadx\config\ram\Crane_DS_8M_Ram_8M_Flash_XIP_CIPSRAM_Common.h 可以清晰看到划分的 RAM 的布局：



实际上 DDR_HEAP_GUARD 到 DDR_RW_DATA 之间是被划分为：

```
|--------------|0x7E070000
|              |
|HEAP          |
|--------------|
|              |
|(APP_RW)      |
|--------------|0x7E4A0000
```

ROM 默认编译生成的固件如下，需要注意 reserved 需要给内部预留至少 150K：



一、内核测裁剪资源：

内核测裁剪文件：ql-kernel\threadx\common\include\quec_cust_feature.h

ql-kernel\threadx\config\common\default_dflags.mk

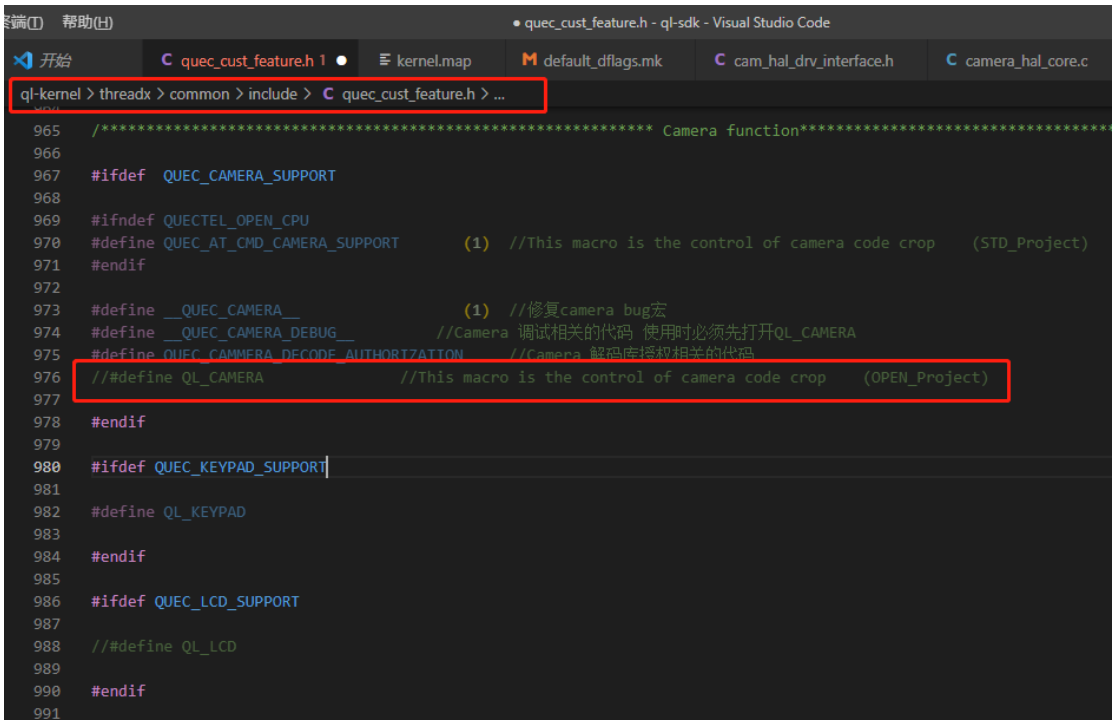
这个文件置起了很多宏定义包括：

-DQUEC_LCD_SUPPORT

-DQUEC_CAMERA_SUPPORT

-DQUEC_KEYPAD_SUPPORT

- SDK 默认裁剪了 CAMERA



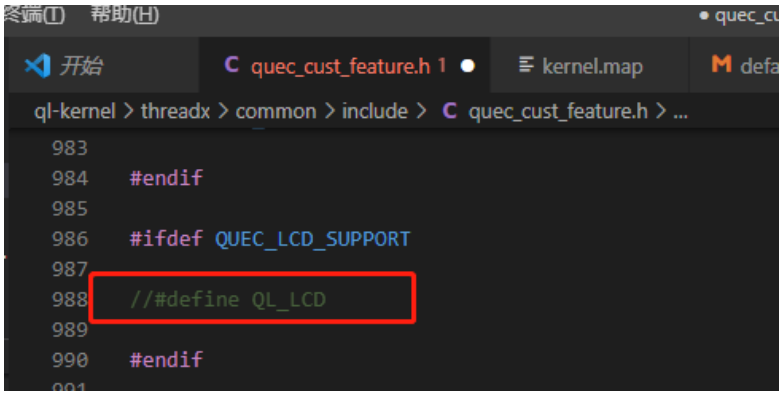
用户若想裁剪 Camera 相关功能，只需屏蔽“QL_CAMERA”就可裁剪掉 camera，若用户需要使用此部分功能只需将“QL_CAMERA”宏定义打开

（注意每次修改头文件中的宏定义，需要执行下 build.bat kernel clean,清理掉之前编译生成的*.o 文件否则修改头文件不起效）

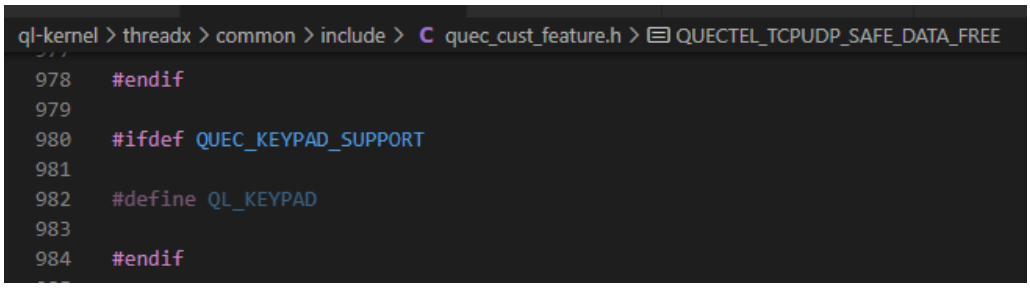
确认裁剪是否生效，可以在 kernel.map 里面查找是否还存在：

打开 ql-sdk\ql-kernel\threadx\build\kernel.map 文件，在 map 表搜索：“CCCamFullSizeStreamThread”相关字样，可以搜到这个函数则说明 camera 代码已编译进去，否则说明已经裁剪，（我们拿到 SDK 默认 QL_CAMERA 该宏是屏蔽的）

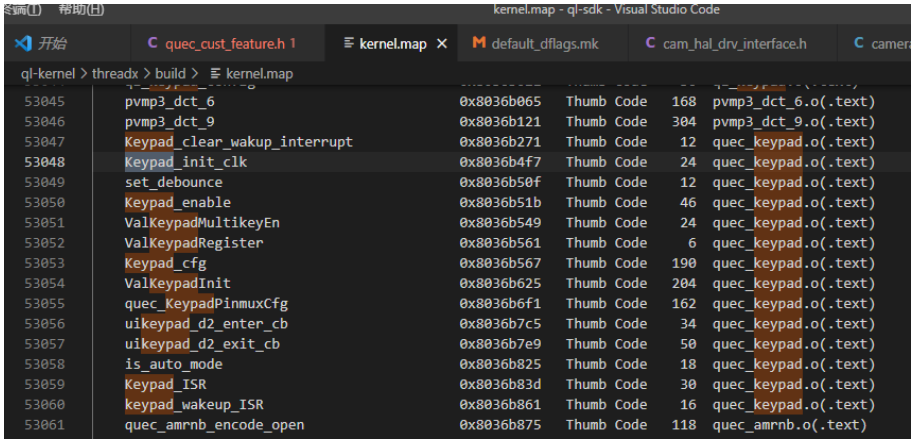
- SDK 默认裁剪了 LCD



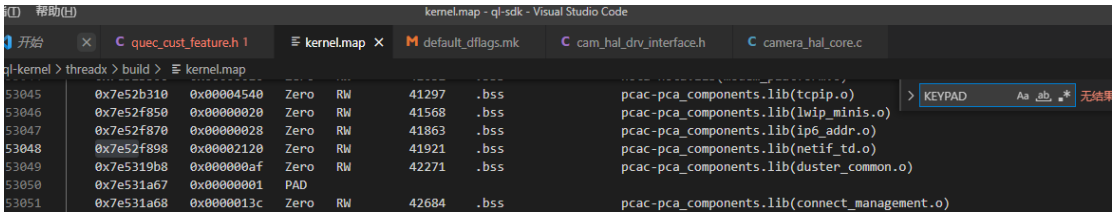
在关闭宏“QL_LCD”，然后执行 build.bat kernel clean 和 build.bat kernel 后，打开 ql-sdk\ql-kernel\threadx\build\kernel.map 文件，在 map 表中搜索“quec_lcd_write_sync”，此函数是内核测写数据接口，map 表中搜索不到此函数即说明 lcd 裁剪成功，反之如果需要添加 QL_LCD 功能，在 map 表搜索该函数，如果存在说明提添加成功.

- SDK 默认开启了 KEYPAD
- 

在关闭宏“QL_KEYPAD”，然后执行 build.bat kernel clean 和 build.bat kernel 后，打开 ql-sdk\ql-kernel\threadx\build\kernel.map 文件，在 map 表中搜索“ValKeypadRegister”，此函数是内核 keypad 注册接口（ql_keypad_Register），map 表中搜索不到此函数即说明 keypad 裁剪成功，反之如果需要添加 QL_KEYPAD 功能，在 map 表搜索该函数，如果存在说明提添加成功.



裁剪之后就搜索不到了

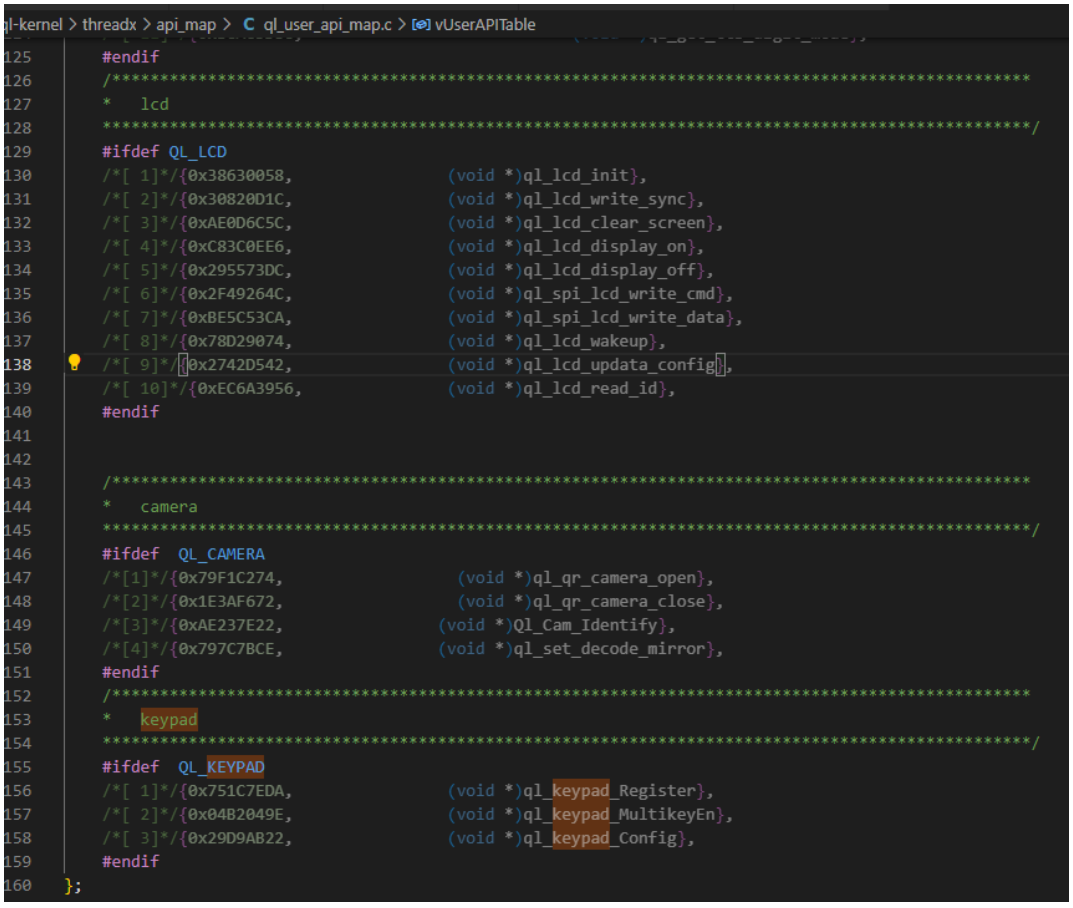


- SDK 默认开启了 TTS

```
如果需要裁剪 TTS，需要宏定义下
#define QUEC_TTS_NOT_SUPPORT

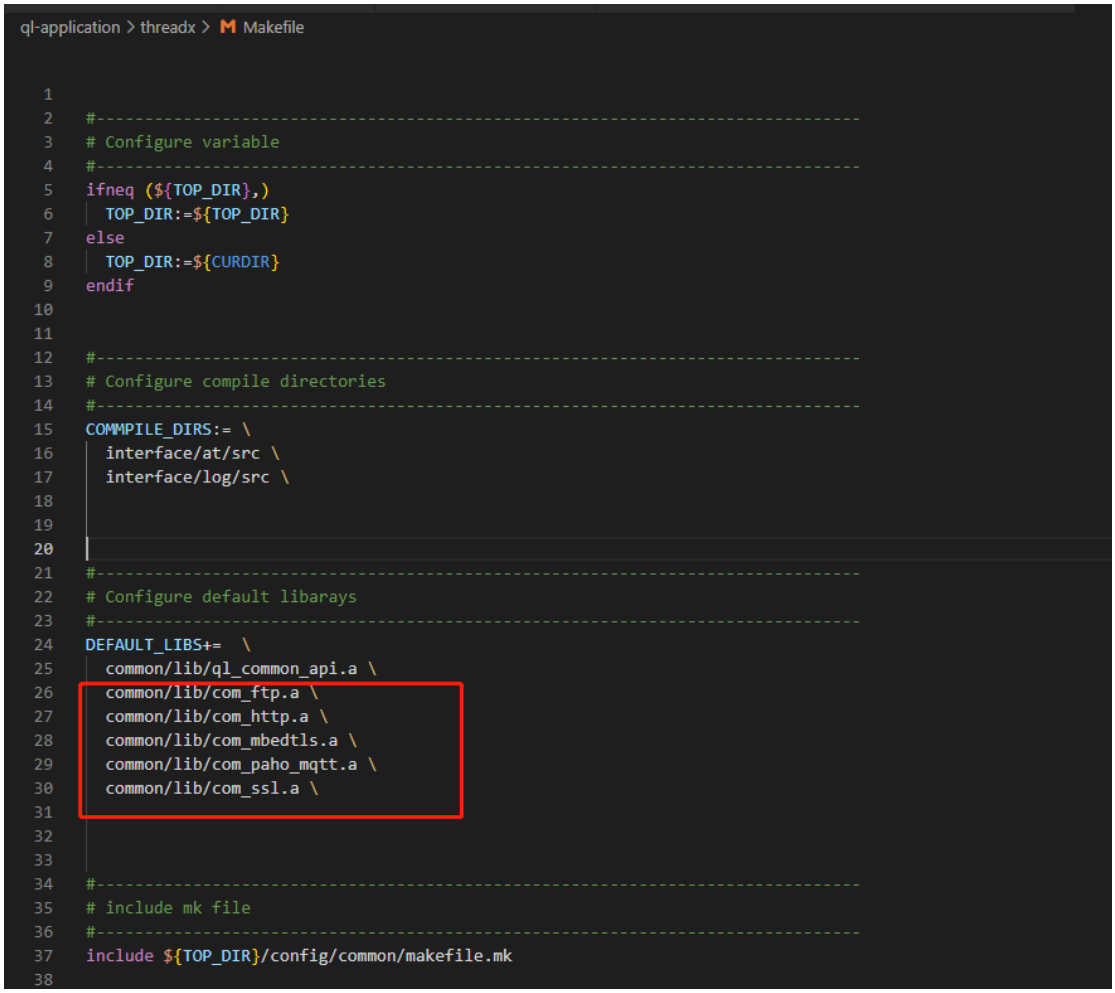
SDK 默认是打开 TTS 功能
#ifndef  QUEC_TTS_NOT_SUPPORT
```

打开之后编译 kernel，ql-kernel\threadx\build\kernel.map 可以搜索看看是否有无这些接口参与编译，从而确认是否有无裁剪：
ql-kernel\threadx\api_map\ql_user_api_map.c 里面的接口比较直观，受到宏控制



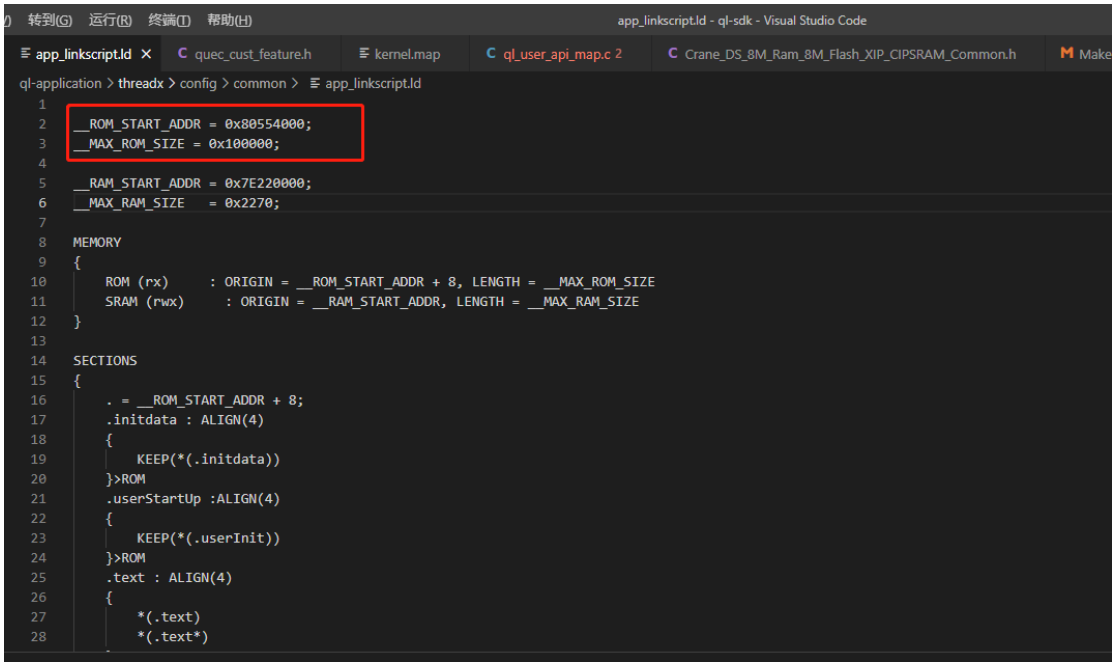
二、应用测裁剪资源：

应用层的第三方库的裁剪比较简单，主要更改下面的文件：
ql-application\threadx\Makefile
可以针对性的把第三方库的 lib.a 删除屏蔽掉就可以了：然后直接编译 app



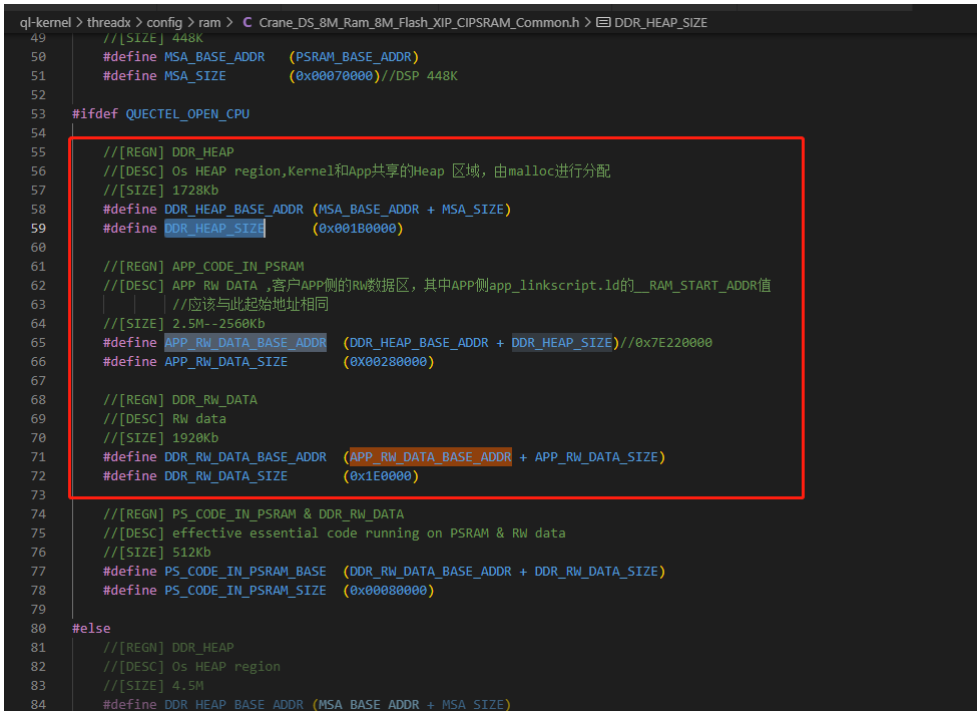
通过编译出来的 ql-application\threadx\build\app.map
里面搜索对应的 lib.a，比如裁剪了 com_ssl.a，如果 map 搜不到了 com_ssl.a 说明裁剪生效了。

无论是内核裁剪还是 APP 测的裁剪，裁剪完了之后一定要关注 customer_app 的地址，通过加载固件到 aboot 工具后查看 customer_app 的地址，此时这个 flash 的地址和大小变动之后，
ql-application\threadx\config\common\app_linkscript.ld
__ROM_START_ADDR = 0x80554000;
__MAX_ROM_SIZE = 0x100000;
这两个值也要对应更改，否则 XIP 跳转的时候 APP 启动不了。



EC600M
三、调整 RAM 布局：

RAM 分配：
ql-sdk\ql-kernel\threadx\config\ram\Crane_DS_8M_Ram_8M_Flash_XIP_CIPSRAM_Common.h
APP RAM 地址
ql-sdk\ql-application\threadx\config\common\app_linkscript.ld
其中,__RAM_START_ADDR 需要和 RAM 分配中的 APP_RW_DATA_BASE_ADDR 匹配



有的客户需要调整 RAM 的大小，会调整 heap 的空间，heap 目前我们是 app 跟 Kernel 一起共用，可以开机注入网络后查询我们的剩余 heap,发 at+qdbgcfg=”memory”,0,2
如下：1727k 是总的，854k 是剩余的

```
[2022-05-29_19:15:59:779]+QDBGCFG: "memory",1727,854,49

[2022-05-29_19:15:59:779]OK
[2022-05-29_19:16:00:033]AT+QDBGCFG="memory",0,2
[2022-05-29_19:16:00:033]+QDBGCFG: "memory",1727,854,49

[2022-05-29_19:16:00:033]OK
[2022-05-29_19:16:12:713]AT+QDBGCFG="memory",0,2
[2022-05-29_19:16:12:713]+QDBGCFG: "memory",1727,853,49

[2022-05-29_19:16:12:713]OK
[2022-05-29_19:16:25:280]AT+QDBGCFG="memory",0,2
[2022-05-29_19:16:25:280]+QDBGCFG: "memory",1727,853,49

[2022-05-29_19:16:25:280]OK
[2022-05-29_19:16:25:729]AT+QDBGCFG="memory",0,2
[2022-05-29_19:16:25:729]+QDBGCFG: "memory",1727,853,49

[2022-05-29_19:16:25:729]OK
```

也可以使用接口：

```
[2022-05-29_16:21:37:922]ql_get_mempool_max_freebuf_size()=834848
[2022-05-29_16:21:39:934]ql_rtos_get_free_heap_size()=889056
[2022-05-29_16:21:39:934]ql_get_mempool_total_size()=1769468
[2022-05-29_16:21:39:934]ql_get_mempool_max_freebuf_size()=834848
[2022-05-29_16:21:41:943]ql_rtos_get_free_heap_size()=890624
[2022-05-29_16:21:41:943]ql_get_mempool_total_size()=1769468
[2022-05-29_16:21:41:943]ql_get_mempool_max_freebuf_size()=834848
[2022-05-29_16:21:43:915]ql_rtos_get_free_heap_size()=891008
[2022-05-29_16:21:43:929]ql_get_mempool_total_size()=1769468
[2022-05-29_16:21:43:929]ql_get_mempool_max_freebuf_size()=834848
[2022-05-29_16:21:45:948]ql_rtos_get_free_heap_size()=891008
[2022-05-29_16:21:45:948]ql_get_mempool_total_size()=1769468
[2022-05-29_16:21:45:948]ql_get_mempool_max_freebuf_size()=834848
[2022-05-29_16:21:47:947]ql_rtos_get_free_heap_size()=891008
[2022-05-29_16:21:47:947]ql_get_mempool_total_size()=1769468
[2022-05-29_16:21:47:947]ql_get_mempool_max_freebuf_size()=834848
[2022-05-29_16:21:49:957]ql_rtos_get_free_heap_size()=891008
[2022-05-29_16:21:49:957]ql_get_mempool_total_size()=1769468
[2022-05-29_16:21:49:957]ql_get_mempool_max_freebuf_size()=834848
```

如果减小了 heap 的 size(DDR_HEAP_SIZE)，那么 app_rw 的 start_addr 就要往上偏移，HEAP+APP_RW 总大小不变，下面很直观

```
|--------------|0x7E070000
|              |
|HEAP          |
|--------------|
|              |
|(APP_RW)      |
|--------------|0x7E4A0000
```

app_rw 的 start_addr 往上偏移之后，那么 ql-application\threadx\config\common\app_linkscript.ld

```
__RAM_START_ADDR = 0x7E220000;
__MAX_RAM_SIZE   = 0x2270;
```

上面的 RAM_START_ADDR 和 SIZE 都要做相应调整

三、调整 ROM 布局：

Flash 分区划分：
ql-sdk\ql-config\quec-project\aboot\config\partition\CRANEL_SINGLE_QUEC_FLASH_LAYOUT_8M_OPEN.json

APP ROM 地址
ql-sdk\ql-application\threadx\config\common\app_linkscript.ld

其中，__ROM_START_ADDR 需要和 aboot 工具见到的地址匹配

